

Engenharia de Software

Aula 01 — Introdução



Prof. Dr. João Choma

Universidade Estadual de Maringá — DIN/CTC

github.com/JoaoChoma

Análise e Projeto de Software

A natureza do software



Ao final desta aula, você será capaz de...

- explicar por que desenvolver software exige mais do que programar;
- reconhecer as principais áreas da Engenharia de Software;
- distinguir requisitos funcionais e não funcionais;
- relacionar defeito, falha, verificação e validação;
- comparar processos sequenciais e iterativos;
- avaliar como contexto e criticidade alteram o rigor necessário.



Um sistema de software combina:

- instruções que serão executadas;
- dados e configurações necessários ao funcionamento;
- documentação para uso, operação e evolução;
- conhecimento sobre regras, decisões e restrições do domínio.

Ideia central

O código perde significado quando é separado do ambiente, dos dados e das pessoas que dependem dele.



Software programas e artefatos associados.

Aplicação software orientado a uma finalidade percebida.

Produto solução mantida para usuários, clientes ou outras partes interessadas.

Infraestrutura sistema operacional, driver, firmware, biblioteca ou serviço que sustenta outros softwares.

A classificação depende de quem observa e de qual papel o software desempenha.



- não sofre desgaste físico pelo uso;
- mudanças podem degradar sua estrutura e criar incompatibilidades;
- copiar uma versão pronta custa pouco, mas projetar e manter custa muito;
- sua estrutura é invisível e precisa ser representada por modelos;
- integrações e adaptações ligam a solução a contextos específicos.

Consequência

O programa pode continuar executando e tornar-se inadequado porque ambiente, dados ou necessidades mudaram.



Tipos de software se sobrepõem

- sistemas e infraestrutura;
- aplicações comerciais;
- software científico;
- sistemas embarcados;
- aplicações web e móveis;
- jogos e entretenimento;
- aprendizado de máquina;
- sistemas legados.

Exemplo

Um automóvel moderno combina software embarcado, tempo real, aplicativo móvel, serviços em nuvem e análise de dados.



O livro de **Marco Tulio Valente** conecta conceitos clássicos a exemplos próximos da prática contemporânea.



Leitura desta aula

Capítulo 1 — Introdução

engsoftmoderna.info/cap1.html

Os slides oferecem o mapa da discussão; o capítulo traz explicações, referências e estudos de caso para aprofundamento.



Quatro perguntas para orientar a leitura

- 1 O que diferencia Engenharia de Software de programação?
- 2 Quais dificuldades são inerentes ao software?
- 3 Como as diferentes áreas da Engenharia de Software se relacionam?
- 4 Por que criticidade e contexto alteram as práticas utilizadas?

Estratégia

Anote um exemplo próprio para cada resposta. Vamos recuperar essas anotações no estudo de caso ao final da aula.



Bancos, hospitais, universidades, governos, transportes e indústrias dependem de software.

Quando ele falha, o impacto pode variar de um pequeno incômodo a prejuízos financeiros ou risco à vida.

Aquecimento

Quantos sistemas de software você utilizou antes de chegar à aula hoje?



- 1 interface web ou aplicativo;
- 2 catálogo e estoque;
- 3 pagamento e antifraude;
- 4 emissão fiscal;
- 5 logística e rastreamento;
- 6 notificações e atendimento.

Onde mora a dificuldade?

Não apenas em cada componente, mas nas relações, exceções e mudanças que atravessam todos eles.



Programação produz instruções executáveis.

Engenharia de Software organiza pessoas, decisões, técnicas e evidências para que a solução seja útil, confiável e sustentável.

Definição de trabalho

Aplicação de abordagens sistemáticas, disciplinadas e mensuráveis ao desenvolvimento, operação, manutenção e evolução de software.



O termo **Engenharia de Software** ganhou projeção na conferência da OTAN de 1968.

A chamada crise do software descrevia projetos cuja escala, complexidade e interdependência ultrapassavam as práticas disponíveis.



Por que existe Engenharia de Software?



“Ferramentas modernas garantem qualidade”

Ferramentas ajudam, mas não substituem entendimento, projeto, revisão, testes e pessoas preparadas.

“Mais pessoas recuperam qualquer atraso”

Aprendizado, comunicação, coordenação e integração também consomem tempo.

O problema de um mito não é simplificar: é esconder condições e custos.



Mito do cliente

“Objetivos gerais bastam; os detalhes aparecem durante o desenvolvimento.”

Nem todo detalhe precisa ser definido antecipadamente. Ainda assim, a equipe precisa compreender:

- necessidades e partes interessadas;
- restrições e riscos relevantes;
- critérios que indiquem sucesso;
- como e quando receberá feedback.

Iterar exige aprendizado frequente — não ausência de direção.



Mito profissional

“Quando o programa funciona e é entregue, o trabalho terminou.”

Depois da entrega, o software precisa continuar útil enquanto mudam:

- necessidades e regras de negócio;
- dados, integrações e plataformas;
- ameaças e requisitos de segurança;
- pessoas que operam e mantêm o sistema.

Operação, suporte, correção e evolução fazem parte da Engenharia de Software.



- Por que estimativas e prazos são difíceis?
- Por que mudanças pequenas podem produzir efeitos grandes?
- Por que não encontramos todos os defeitos antes da entrega?
- Como saber se o projeto está realmente progredindo?
- Como manter qualidade enquanto sistema e equipe evoluem?

Resposta realista

A disciplina não elimina a complexidade; oferece princípios, práticas e evidências para tratá-la conscientemente.



Não existe uma “bala de prata”

Frederick Brooks argumentou que nenhuma tecnologia isolada eliminaria as dificuldades essenciais do desenvolvimento de software.

- complexidade;
- conformidade;
- mudança contínua;
- invisibilidade da estrutura.





A complexidade aparece nas dependências

Solicitação aparentemente simples

“Adicionar um cupom de desconto ao checkout.”

Isso pode alterar preços, impostos, campanhas, estoque, pagamento, cancelamento, relatórios e prevenção de fraude.

O tamanho da mudança no código não mede o tamanho do problema.

O mapa da disciplina



As áreas formam um sistema de trabalho

- requisitos;
- projeto e arquitetura;
- construção;
- testes e qualidade;
- manutenção e evolução;
- gerência de configuração;
- gerência de projetos;
- processos e modelagem;
- prática profissional e ética;
- aspectos econômicos.

Importante

Em processos iterativos, essas áreas reaparecem continuamente — não são departamentos nem fases completamente isoladas.



Requisitos registram necessidades, objetivos, comportamentos, restrições e qualidades esperadas.

A Engenharia de Requisitos inclui:

- descobrir e negociar necessidades;
- analisar conflitos e dependências;
- documentar de forma adequada ao contexto;
- validar o entendimento com as partes interessadas.



Requisito funcional

“O cliente deve poder redefinir sua senha.”

Define um serviço ou comportamento do sistema.

Requisitos não funcionais

O link expira em 15 minutos.

95% das respostas em até 2 segundos.

Dados sensíveis são criptografados.

Define qualidade ou restrição de operação.



Versão inicial

O aplicativo deve ser rápido e seguro.

Perguntas que transformam intenção em critério

Para qual operação? Com quantos usuários? Qual percentil? Que dados precisam de proteção? Contra quais ameaças? Como a conformidade será demonstrada?

Um bom requisito reduz interpretações incompatíveis.



Análise quais conceitos, regras e responsabilidades existem no problema?

Projeto como componentes, interfaces, dados e arquitetura organizarão a solução?

Construção como implementar, integrar e executar essas decisões?

Exemplo: entrega de pedidos

Pedido e pagamento são conceitos; APIs e mensageria são decisões de projeto; código, configuração e testes materializam a solução.

Qualidade ao longo do ciclo de vida

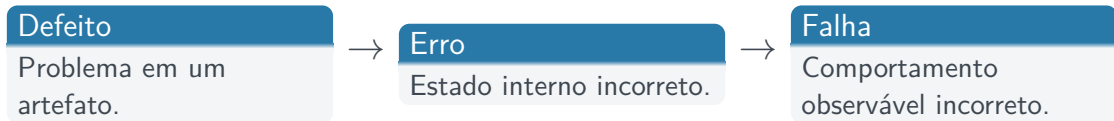


Testar busca diferenças entre o comportamento observado e o esperado.

Dijkstra

Testes podem revelar a presença de defeitos, mas não demonstram sua ausência.

A estratégia combina níveis, técnicas, automação, revisão e análise de risco.



Um defeito pode permanecer oculto até que uma entrada ou condição específica o ative.



O caso de 1996 envolveu a conversão de um valor real de 64 bits para um inteiro de 16 bits na cadeia que levou à falha.

- reuso exige revalidar hipóteses;
- faixas e unidades devem ser explícitas;
- exceções fazem parte do comportamento;
- redundância não remove defeitos idênticos.



Verificação

Estamos construindo corretamente, de acordo com requisitos e especificações?

Validação

Estamos construindo a solução que atende à necessidade real?

Um sistema pode cumprir a especificação e ainda fracassar para o usuário.



Corretiva remove defeitos.

Adaptativa responde a mudanças de ambiente.

Evolutiva adiciona ou altera capacidades.

Preventiva reduz riscos futuros.

Refatoração melhora a estrutura sem mudar o comportamento observável.

Manutenibilidade é uma propriedade econômica, não apenas estética.



Muitos sistemas representavam o ano com dois dígitos. A virada para 2000 exigiu localizar, corrigir, testar e coordenar dependências.

O desafio não era trocar “AA” por “AAAA”, mas descobrir onde essa hipótese estava embutida.





Sistemas legados frequentemente concentram conhecimento de negócio e sustentam operações essenciais.

Decisões possíveis

Modernizar gradualmente, encapsular, substituir, reescrever ou manter com controles adicionais.

A escolha depende de risco, custo, conhecimento disponível, compatibilidade e capacidade de testar o comportamento existente.



Gerência de configuração

Mantém versões confiáveis de código, documentos, dependências, ambientes e releases.

Gerência de projetos

Coordena escopo, pessoas, riscos, custos, comunicação e entregas.

Git é parte da gerência de configuração — não seu sinônimo.



Adicionar pessoas a um projeto atrasado pode atrasá-lo ainda mais.

- novos integrantes precisam aprender o contexto;
- comunicação e coordenação crescem;
- tarefas nem sempre podem ser divididas;
- integração adicional cria novos riscos.

Leitura correta

Não proíbe ampliar equipes; alerta que pessoas não são unidades de capacidade intercambiáveis.

Processos, modelos e decisões



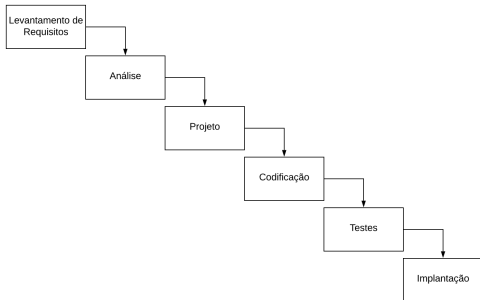
Um processo define atividades, responsabilidades, artefatos e critérios usados para desenvolver e evoluir software.

Sequencial

Grandes fases são concluídas antes do avanço.

Iterativo e incremental

A solução evolui por ciclos e entregas parciais.



Funciona melhor quando:

- objetivos são estáveis;
- o domínio é conhecido;
- mudanças são controladas;
- evidências formais são exigidas.

O risco cresce quando o aprendizado acontece apenas no final.



Incremental adiciona capacidades utilizáveis ao produto.

Iterativo revisa a solução com base em feedback e aprendizado.

Exemplo: loja on-line

Catálogo → carrinho → pagamento → rastreamento → recomendações.

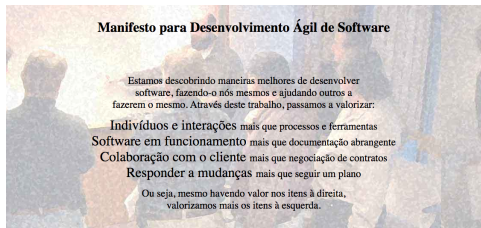
Uma entrega pequena só ajuda quando produz valor ou informação para a próxima decisão.



O Manifesto Ágil enfatiza:

- pessoas e interações;
- software funcionando;
- colaboração com o cliente;
- resposta a mudanças.

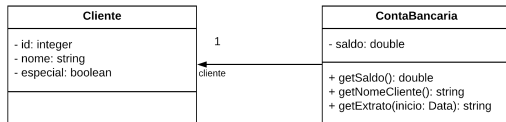
Os itens à direita continuam tendo valor; apenas não recebem prioridade quando entram em conflito.





Um modelo omite detalhes para destacar uma decisão.

- comunica estrutura;
- analisa alternativas;
- registra decisões;
- apoia manutenção.



O melhor modelo não é o mais detalhado, mas o mais útil.



Qualidade externa

Correção, desempenho, segurança, disponibilidade e usabilidade.

Qualidade interna

Modularidade, legibilidade, testabilidade e manutenibilidade.

Segurança pode adicionar atrito; desempenho pode aumentar complexidade; velocidade pode gerar dívida técnica. Engenharia torna as escolhas explícitas.



Ética

Privacidade, acessibilidade, segurança, transparência e impacto social.

Economia

Custo de oportunidade, retorno, licenças, receita e manutenção.

Discussão: uma funcionalidade que aumenta conversão ao induzir uma escolha desinformada é um sucesso?

Risco e síntese



C — Casual

Pequeno impacto e vida curta.

B — Business

Importante para uma organização.

A — Acute

Falhas podem causar perdas extremas.

Evidências, revisões, testes, rastreabilidade e certificações aumentam conforme cresce o impacto potencial.



Transporte



Infraestrutura



Saúde

O mesmo recurso pode exigir rigor diferente conforme seu papel no sistema.



Em grupos, projete a primeira versão de uma agenda para uma clínica:

- 1 defina três requisitos funcionais;
- 2 escreva dois requisitos não funcionais mensuráveis;
- 3 identifique uma decisão de projeto;
- 4 proponha um teste que revele uma falha relevante;
- 5 classifique a criticidade e justifique;
- 6 escolha uma primeira entrega incremental útil.



- VALENTE, Marco Tulio. *Engenharia de Software Moderna*, cap. 1. engsoftmoderna.info/cap1.html
- NAUR, Peter; RANDALL, Brian (eds.). *Software Engineering: Report of a Conference Sponsored by the NATO Science Committee*, 1968.
- BROOKS, Frederick P. *No Silver Bullet: Essence and Accidents of Software Engineering*. IEEE Computer, 1987.
- BECK, Kent et al. *Manifesto para Desenvolvimento Ágil de Software*, 2001. agilemanifesto.org/iso/ptbr/manifesto.html